

Figure C.8 Insertion in a linked list in any position. In this example, we show insertion on a sorted linked list (but sorting is not a requirement). (A) The original list. (B) Walk the linked list till we find the predecessor of the position where we want to insert the new element (being a singly linked list, we need to maintain a pointer to the predecessor during this search). Let's denote this predecessor with `P`. (C) Create a new node `N`, and update the next pointer for `P`. At the same time, set `N.next` to `P.next`. (D) Final result.

Similarly, we can double down the same reasoning for node removal:

- **From the beginning**.

  It can be done in constant time by updating the head reference to the old head's successor (after checking the list isn't empty). And we should better make sure the removed node is deallocated, or made available for garbage collection.

- **From the end**.

  We need to find the second to last node and update its pointer to the next node. For doubly linked list we just get to the end of the list and then take its predecessor. For singly linked lists we need to keep a reference not just to current node and to its predecessor while we traverse the list. This is a linear time operation.

- **From any other position**.

  Same considerations as for insertions, plus the concern for keeping a reference to the second last element in the list.

Linked lists are recursive data structures. This steams out of their recursive definition, and it means that problems on lists can be reasoned about by induction. You can (try to) provide a solution for the base case (the empty list) and a way to combine some actions on the head of the